

NIKE STORE

Table of Contents:

Introduction	3
Database Design	5
Minimum Cardinality Tables	6
Default Values Table	10

Description:

NIKE was established in 1964, since then, NIKE has expanded and grown exponentially as one of the top shoe and sports clothing in the world. As Nike constantly grows and expands, the managers Stefano Yannitsaros and Fragiskos Parson have been given the task to integrate the database into an efficient and effective system whose goal is to increase productivity, save money and make the general fluidity of the business better.

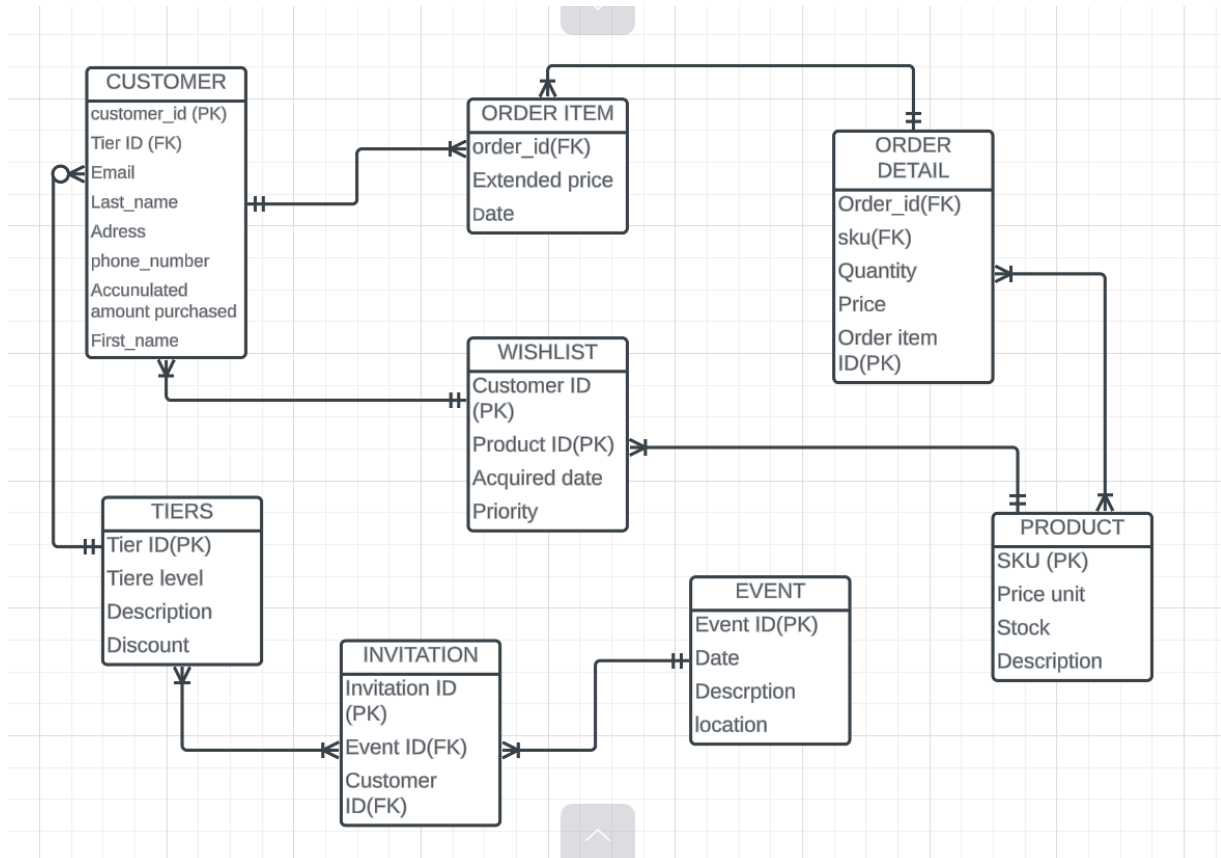
In the online database, customers can sign up for a customer loyalty program that utilizes points in order to provide them with discounts, gifts, vouchers, events and many more. The aim of the program is to provide value to the customer that routinely purchases products from our store. We have included 4 tiers in order to differentiate and group our customers based on the frequency of purchases in our store. Customers can use their points and tier benefits to acquire products from their wishlist. Tier 1 grants a 10% discount to customers spending less than \$200 in a 3 month period. Tier 2 includes customers who have been spending between \$200 and \$500, which grants 15% discount. Tier 3 grants customers 25% and the ability to redeem 1 wishlist item per season. Tier 4 grants customers the ability to attend special events that are hosted by Nike, redeem one wishlist item per season, and additionally the 25% discount. In order to fall within tier 4 (tier 3 requirements must be met), customers need to be a diamond member on the Nike App for Health, and be enrolled in the program for more than 3 years.

The company also needs the ability to track their inventory, thus the database needs to include a data table PRODUCT which has the attributes of SKU, stock, price and more.

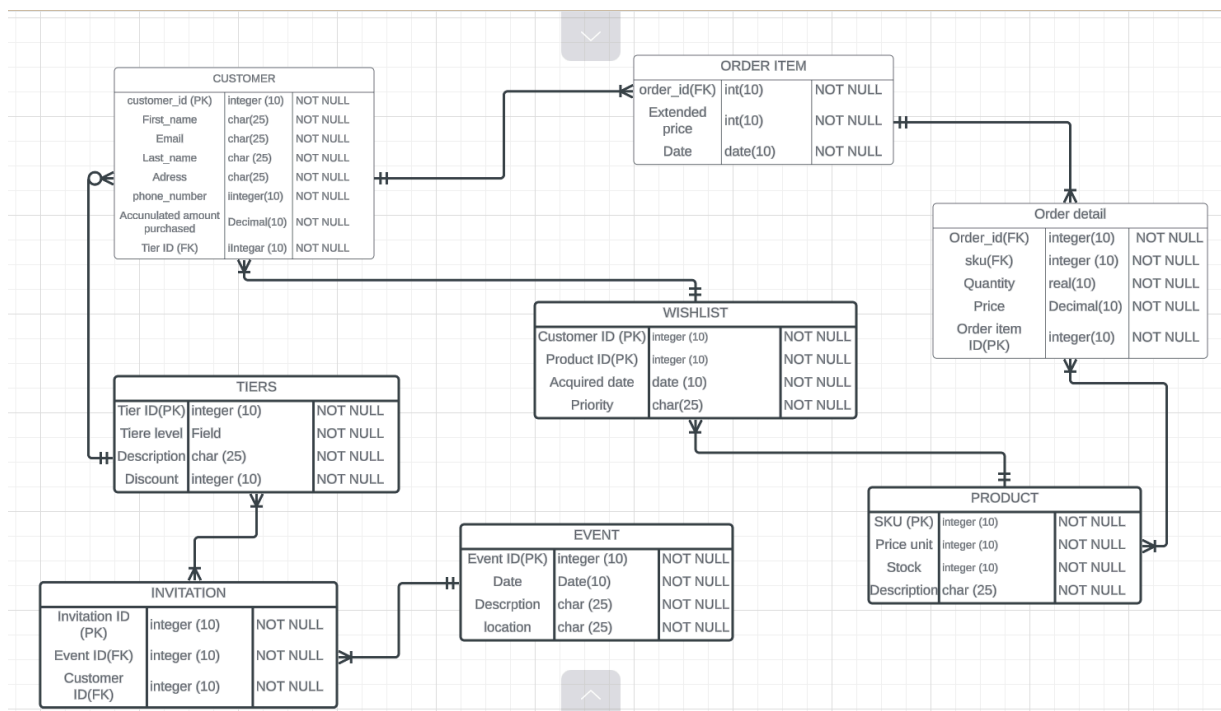
The customer can sign up for the customer loyalty program during the payment process. Our staff is trained in order to ask and navigate the customer to provide the necessary information in order for the database to work. The customer has to add first name, last name, email, address, phone number. A customer ID, wishlist ID and passwords, which will allow the user to login whenever they visit the store. If the client doesn't want to sign up for our program, the account will not be used, since its optional.

Database Designs:

ER diagram with attributes and primary keys



ER diagram with all keys and attributes



Minimum Cardinality Tables:

- Customer Loyalty (Parent)
- Event (Child)

	Action On Parent	Action on Child
Insert	None	Get a Customer Loyalty ID(Parent)
Modify	Prohibit - Uses surrogate key. (ID)	Prohibit - Uses surrogate key. (ID)
Delete	Prohibit - Unless the customer wants to delete their data.	None

Business Rule: Unless the Event is canceled and never rescheduled

- Wishlist (Parent)
- Customer Loyalty(Child)

	Action On Parent	Action on Child
Insert	None	Get a parent. (Wishlist ID)
Modify	Prohibit - Use of surrogate Key (ID)	Prohibit - Use of surrogate Key(ID)
Delete	Prohibit - Unless the customer wants to delete their data.	None

- Product (Parent)
- Wishlist (Child)

	Action On Parent	Action on Child
Insert	None	Get a parent. (ProductID)
Modify	Prohibit - Use of surrogate Key (ID)	Prohibit - Use of surrogate Key(ID)
Delete	Prohibit - Unless the customer wants to delete their data.	None

- Product (Parent)
- Order_Item (Child)

	Action On Parent	Action on Child
Insert	None	Get a parent. (Price ID)
Modify	Prohibit - Use of surrogate Key (ID)	Prohibit - Use of surrogate Key(ID)
Delete	Prohibit - Unless Delete Product from inventory	Do not show any instance of the product.

Business Rule: Unless the product is discontinued due to supply etc.

- Order (Parent)
- Order Item (Child)

	Action On Parent	Action on Child
Insert	None	Get a parent. (Order ID)
Modify	Prohibit - Use of surrogate Key (ID)	Prohibit - Use of surrogate Key(ID)
Delete	Prohibit - Unless the customer wants to delete their data.	None

Business Rule: Unless the customer wants to remove their order.

- Customer (Parent)
- Payment (Child)

	Action On Parent	Action on Child
Insert	None	Get a parent. (Customer ID)
Modify	Prohibit - Use of surrogate Key (ID)	Prohibit - Use of surrogate Key(ID)
Delete	Prohibit - Unless the customer wants to delete their data.	None

Business Rule: Unless the payment method needs to be deleted due to theft/loss.

- Customer(Parent)
- Customer Loyalty (Child)

	Action On Parent	Action on Child
Insert	None	Get a parent. (Customer ID)
Modify	Prohibit - Use of surrogate Key (ID)	Prohibit - Use of surrogate Key(ID)
Delete	Prohibit - Unless the customer wants to delete their data.	None

Business Rule: If customer requests to delete their data (GDPR)

Table Default Values:

Table	Column	Default Value
Tiers	Discount	None
Tiers	Tier	None
Tiers	ID	Primary Key
Events	Name	None
Events	Date	None
Events	Description	None
Events	Location	None
Events	CustomerLoyalty	Foreign Key
Invitations	Event Id	Primary Key
Invitations	Name	None
Invitations	Date	None
Invitations	Description	None
Invitations	Location	None
Wishlist	WishlistID	Primary Key
Wishlist	Product ID	Foreign Key
Wishlist	Customer ID	Foreign Key
Wishlist	AcquiredDate	None
Wishlist	Priority	None
Customer	Customer ID	Primary Key
Customer	First Name	None

Customer	Last Name	None
Customer	Email	None
Customer	Address	None
Customer	Phone Number	None
Product	SKU	Primary Key
Product	Description	None
Product	Price	AK(1.1)
Product	Stock	None
Order	Order ID	Primary Key
Order	Oder Date	None
Order	Total Price	None
Order	Quantity	None
OrderDetail	Order ID	Foreign Key
OrderDetail	Quantity	None
OrderDetail	SKU	Foreign Key

Tables:

❖ **CREATE TABLE EVENTS**

```
CREATE TABLE Events (  
    EventID INT NOT NULL,  
    [Date] DATE NOT NULL,  
    Description VARCHAR(255) NOT NULL,  
    Location VARCHAR(50) NOT NULL,  
    CONSTRAINT EventPK PRIMARY KEY (EventID)  
);
```

❖ **CREATE TABLE INVITATIONS**

```
CREATE TABLE Invitations(  
    InvitationID INT NOT NULL,  
    EventsID INT NOT NULL,  
    TierID INT NOT NULL,  
    CONSTRAINT InvitationsPK PRIMARY KEY (InvitationID)  
    CONSTRAINT EventsFK FOREIGN KEY (EventsID) REFERENCES Events(EventsID)  
    CONSTRAINT TierFK FOREIGN KEY (TierID) REFERENCES Tiers(TierID)  
);
```

❖ **CREATE TABLE Tiers (**

```
TierID INT NOT NULL,  
TierLevel INT NOT NULL,  
Description Char(200) NOT NULL,  
Discount DECIMAL(5,2) NOT NULL,  
CONSTRAINT TiersPK PRIMARY KEY (TierID)
```

);

❖ **CREATE TABLE Customer (**

CustomerID INT NOT NULL,
FirstName Char(50) NOT NULL,
LastName Char(50) NOT NULL,
Email Char(100) NOT NULL,
Address Char(200) NOT NULL,
AccumulatedAmountPurchased DECIMAL(10,2) NOT NULL,
TierID INT NOT NULL,
CONSTRAINT CustomerPK PRIMARY KEY (CustomerID),
CONSTRAINT TierFK FOREIGN KEY (TierID) REFERENCES Tiers(TierID)
CONSTRAINT CustmerAK UNIQUE(Email)
);

❖ **CREATE TABLE Orders (**

OrderID INT NOT NULL,
CustomerID INT NOT NULL,
OrderDate DATE NOT NULL,
TotalPrice DECIMAL(10,2) NOT NULL,
CONSTRAINT OrdersPK PRIMARY KEY (OrderID),
CONSTRAINT CustomerFK FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

❖ **CREATE TABLE ProductDetail (**

ProductDetailID INT NOT NULL,
ProductID Char(50) NOT NULL,
Description Char(200) NOT NULL,
CONSTRAINT ProductDetailsPK PRIMARY KEY (ProductDetailID),
CONSTRAINT PrdocutFK FOREIGN KEY (ProductID) REFERENCES Products(SKU)
);

❖ **CREATE TABLE Products(**

SKU Char(50) NOT NULL,
Description Char(250) NOT NULL,
Price DECIMAL(10, 2) NOT NULL,
Stock INT NOT NULL,
CONSTRAINT ProductsPK PRIMARY KEY (SKU)
);

❖ **CREATE TABLE Wishlist(**

```
WishlistID INT NOT NULL,  
CustomerID INT NOT NULL,  
ProductsID Char(50) NOT NULL,  
CONSTRAINT WishlistPK PRIMARY KEY (WishlistID),  
CONSTRAINT CustomerFK FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
CONSTRAINT ProductFK FOREIGN KEY (Product ID) REFERENCES Products(SKU)  
);
```

Insert records into table, after their creation:

❖ **INSERT RECORDS INTO TABLE EVENTS**

```
INSERT INTO Events (EventID, Date, Description, Location)  
VALUES (011, '15-03-2023', 'Summer Sale Event', 'Nike Store Paris');  
VALUES (021, '11-03-2023', 'Usain Bolt Event', 'Nike Store los angeles');  
VALUES (023, '1-08-2023', 'Rainbow Event', 'Nike Store Amsterdam');  
VALUES (036, '19-03-2023', 'Christmas event', 'Nike Store London');  
VALUES (077, '27-06-2023', 'Kylian Mbappe event', 'Nike Store Franshuke');  
VALUES (034, '18-02-2023', 'Basketball shoot event', 'Nike Store Senderwood');  
VALUES (045, '24-11-2023', 'Jamaican national event', 'Nike Store Indonesia');  
VALUES (187, '02-03-2023', 'Summer sale event', 'Nike Store Uruguay');  
VALUES (139, '26-02-2023', 'Winter Sale event', 'Nike Store Marlboro');  
VALUES (134, '11-10-2023', 'Europe big sale', 'Nike Store New Orleans');
```

❖ **INSERT RECORDS INTO TABLE INVITATIONS**

```
INSERT INTO Invitations (InvitationID, EventID, TierID)  
VALUES (21, 011, 04-058);  
VALUES (22, 021, 06-034);  
VALUES (32, 023, 03-023);  
VALUES (33, 036, 06-078);  
VALUES (38, 077, 03-077);  
VALUES (42, 034, 08-045);  
VALUES (87, 045, 05-063);  
VALUES (90, 187, 07-089);  
VALUES (25, 139, 03-065);  
VALUES (28, 134, 06-039);
```

❖ **INSERT RECORDS INTO TABLE TIERS**

```
INSERT INTO Tiers (TierID, TierLevel, Description, Discount)
VALUES (04-058, 2, 'SILVER', 0.15);
VALUES (06-034, 3, 'GOLD', 0.20);
VALUES (03-023, 4, 'PLATINUM', 0.30);
VALUES (06-078, 2, 'SILVER', 0.15);
VALUES (03-077, 2, 'SILVER', 0.15);
VALUES (08-045, 4, 'PLATINUM', 0.30);
VALUES (05-063, 3, 'GOLD', 0.20);
VALUES (07-089, 3, 'GOLD', 0.20);
VALUES (03-065, 4, 'PLATINUM', 0.30);
VALUES (06-038, 1, 'BRONZE', 0.10);
```

❖ **INSERT RECORDS INTO TABLE CUSTOMER**

```
INSERT INTO Customers (CustomerID, FirstName, LastName, Email, Address, PhoneNumber,
AccumulatedAmountPurchased, TierID)
VALUES (01001, 'John', 'Papas', 'johnpapas@gmail.com', 'Dampon 26', '03736433', 04-058);
VALUES (01201, 'Frank', 'Bolt', 'Frankbol@gmail.com', 'Senderwood 45', '02756642', 06-034);
VALUES (01031, 'Paul', 'Harris', 'Paulharris@gmail.com', 'Glenwood 39', '0373342', 03-023);
VALUES (01231, 'Joanna', 'mark', 'Joannamark@gmail.com', 'Houston 15', '04736342', 06-078);
VALUES (01061, 'Peter', 'Tom', 'Petertom@gmail.com', 'Plett bay 03', '05739682', 03-077);
VALUES (01901, 'Fransi', 'Rich', 'Fransirich@gmail.com', 'glenacker 26', '03976582', 08-045);
VALUES (01891, 'Stef', 'Yannis', 'Stefyannis@gmail.com', 'ocean bridge 2', '03934582', 05-063);
VALUES (01941, 'Adam', 'haralom', 'Adamhara@gmail.com', 'marcia road', '02733492', 07-089);
VALUES (01861, 'Michael', 'joana', 'Michaeljo@gmail.com', 'DownNorth 8', '03735452', 03-065);
VALUES (01441, 'Luis', 'haul', 'Luishaul@gmail.com', 'Bedfordview 12', '03736567', 06-039);
```

❖ **INSERT RECORDS INTO TABLE ORDER**

```
INSERT INTO Orders (OrderID, CustomerID, OrderDate, TotalPrice)
VALUES (01223, 01001, '11-02-2022', 350);
VALUES (01234, 01201, '10-23-2023', 550);
VALUES (02454, 01031, '08-22-2022', 330);
VALUES (03454, 01231, '10-30-2021', 390);
VALUES (01244, 01061, '12-11-2023', 750);
VALUES (21434, 01901, '02-23-2022', 380);
VALUES (43234, 01891, '02-08-2022', 450);
VALUES (34543, 01941, '11-22-2023', 650);
VALUES (51451, 01861, '10-06-2022', 270);
VALUES (54315, 01441, '02-01-2022', 780);
```

❖ **INSERT RECORDS INTO TABLE ORDER_DETAIL**

```
INSERT INTO Order_Detail (OrderDetailID, OrderID, ProductID, Quantity, Price)
VALUES (23, 01223, 'U0DBB-ELE-04' , 4 , '59,99');
VALUES (25, 01234, 'U1DEE-ELE-03' , 3 , '119,9');
VALUES (14, 02454, 'U1DEE-ELE-04' , 4 , '49,99');
VALUES (17, 03454, 'U0DTT-ELE-03' , 3 , '49,99');
VALUES (29, 01244, 'U1DBB-ELE-06' , 6 , '89,99');
VALUES (35, 21434, 'U0DTT-ELE-05' , 5 , '69,99');
VALUES (39, 43234, 'U0DBB-ELE-06' , 6 , '99,99');
VALUES (11, 34543, 'U1DTT-ELE-05' , 5 , '59,99');
VALUES (24, 51451, 'U0DBB-ELE-03' , 3 , '49,99');
VALUES (38, 54315, 'U0DBB-ELE-05' , 5 , '69,99');
```

❖ **INSERT RECORDS INTO TABLE PRODUCTS**

```
INSERT INTO Products (SKU, Description, Price, Stock)
VALUES ('UGCBB-AIR-02', 'Mens shoe Air Force 1 red', '59,99', 36);
VALUES ('UGCBB-AIR-01', 'Mens shoe Air Force 1 Black', '119,9', 30);
VALUES ('UGCBB-AIR-34', 'Mens shoe Air Force 1 White', '49,99', 43);
VALUES ('UGCBB-AIR-01', 'Mens shoe Air Force 1 Blue', '119,9', 28);
VALUES ('URCBB-RUN-03', 'Mens shoe Nike runners Green', '89,99', 36);
VALUES ('URCBB-RUN-04', 'Mens shoe Nike runners Red', '69,99', 30);
VALUES ('UDCBB-DUNK-01', 'Mens shoe Nike Dunk White', '99,99', 43);
VALUES ('UDCBB-DUNK-09', 'Mens shoe Nike Dunk Orange', '59,99', 30);
VALES ('UDCBB-DUNK-19', 'Mens shoe Nike Dunk Green', '49,99', 36);
VALES ('UGCBB-Flop-35', 'Mens shoe Nike Flops Black', '69,99', 28);
```

❖ **INSERT RECORDS INTO TABLE WISHLIST**

```
INSERT INTO Wishlist (WishlistID, CustomerID, ProductID)
VALUES (20034,01001 , 'U0DBB-ELE-04');
VALUES (20014, 01201, 'U1DEE-ELE-03');
VALUES (20034, 01031, 'U1DEE-ELE-04');
VALUES (20234, 01231, 'U0DTT-ELE-03');
VALUES (20134, 01061, 'U1DBB-ELE-06');
VALUES (20500, 01901, 'U0DTT-ELE-05');
VALUES (20074, 01891, 'U0DBB-ELE-06');
VALUES (20004, 01941, 'U1DTT-ELE-05');
VALUES (20100, 01861, 'U0DBB-ELE-03');
VALUES (20200, 01441, 'U0DBB-ELE-05');
```

QUERY 1:

Extract common data from the customer and tier tables, show the records.

```
SELECT C.CustomerID, C.LastName, C.AccumulatedAmountPurchased, T.TierLevel
FROM Customers Tier
```

WHERE Customer.TierID = Tier.TierID;

INSERT PHOTO

QUERY 2:

Extract all data from events that have special guest

SELECT *

FROM Events

WHERE Description LIKE '%Special Guest%';

INSERT PHOTO

QUERY 3:

Count all the fields in the tier table, and display the result as AmountOfLoyalCustomers

SELECT TierID, COUNT(*) AS AmountOfLoyalCustomers

FROM Tiers

GROUP BY TierID;

INSERT PHOTO

QUERY 4:

Select all common data where the priority of a customer in the Wishlist Table, is URGENT.

SELECT Wishlist.WishlistID, Customer.CustomerID, Product.ProductID, Wishlist.Priority

FROM Wishlist , Customers , Products

WHERE Wishlist.CustomerID = Customer.CustomerID AND Wishlist.ProductID = Product.SKU AND
Wishlist.Priority = 'URGENT';

INSERT PHOTO

QUERY 5:

Show orders that have an excess extended price of 200, from table ORDER.

SELECT Order.OrderID, Order.OrderDate, OrderDetails.ExtendedPrice

FROM Orders , OrderDetails

WHERE Order.OrderID = OrderDetails.OrderID AND OrderDetails.ExtendedPrice > 200;

INSERT PHOTO

QUERY 6:

Find customer switch high accumulated purchases in a specific tier

SELECT Customer.CustomerID, Customer.FirstName, Customer.LastName,

Customer.AccumulatedAmountPurchased

FROM Customers


```
WHERE Customer.TierID = (  
    SELECT Tiers.TierID  
    FROM Tiers  
    WHERE Tiers.TierLevel = 4  
);
```

INSERT PHOTO

QUERY 7:

Identify customer with high wishlist priority and tier discount

```
SELECT Customer.CustomerID, Customer.FirstName, Customer.LastName, W.Priority, T.Discount  
FROM Customers  
JOIN Wishlist ON Customer.CustomerID = Wishlist.CustomerID  
JOIN Tiers ON Customer.TierID = Tiers.TierID  
WHERE Wishlist.Priority = 'URGENT' AND Tier.Discount > 0.1
```

INSERT PHOTO

QUERY 8:

Find products in wishlist with low stock(YOU CAN REPHRASE THIS, MAKE IT SOUND NICE)

```
SELECT Wishlist.WishlistID, Products.ProductID, Products.Description, Products.Stock  
FROM Wishlist  
JOIN Products ON Wishlist.ProductID = Products.SKU  
WHERE Products.Stock < 10;
```

INSERT PHOTO

QUERY 9:

Calculate total revenue from orders in the last month(YOU CAN REPHRASE THIS, MAKE IT SOUND NICE)

```
SELECT SUM(ORDER.ExtendedPrice) AS TotalRevenue  
FROM OrderDetails  
JOIN Orders ON ORDER.OrderID = ORDER.OrderID  
WHERE ORDER.OrderDate >= DATEADD(MONTH, -1, 18/12/2023);
```

INSERT PHOTO

QUERY 10:

Find customers eligible for a special event (YOU CAN REPHRASE THIS, MAKE IT SOUND NICE)

```
SELECT Customer.CustomerID, Customer.FirstName, Customer.LastName, E.EventID, E.Description  
FROM Customers  
JOIN Tiers ON Customer.TierID = Tier.TierID  
JOIN Invitations ON Tier.TierID = Invitation.TierID  
JOIN Events ON Invitation.EventID = Event.EventID
```

```
WHERE Tier.TierLevel = 4;  
INSERT PHOTO
```